DBMS LABORATORY

Complete Solutions for All 14 Problem Statements

SQL • PL/SQL • Cursors • Stored Procedures • Triggers • Views • Indexes • MongoDB • Database Connectivity

Prepared from the uploaded DBMSL practical problem statements.

SQL examples primarily use Oracle syntax. MongoDB examples use mongosh syntax. The database-connectivity example uses Python with MySQL Connector.

Before execution: run each problem statement in a separate schema/session or drop existing tables first. Some statements intentionally use Oracle-specific commands such as MINUS, INTERSECT, PL/SQL and DBMS_OUTPUT.

How to use this solution book

Each problem statement includes table creation and sample data where required, followed by solutions to the listed tasks. The supplied problem sheet repeats similar schemas; repeated statements are kept separate so you can prepare from the exact practical number.

For SQL tasks, execute the CREATE TABLE and INSERT statements first, then run the numbered queries. For PL/SQL, enable output with **SET SERVEROUTPUT ON** in SQL*Plus / SQL Developer. Run each PL/SQL block with a slash on a new line when using SQL*Plus.

Foreign keys normally prevent deleting a parent customer who still has account or loan rows. For delete questions, either delete dependent rows first or use a fresh test dataset. The examples below call out this issue where relevant.

Oracle vs MySQL

The uploaded sheet includes Oracle-only features (PL/SQL, MINUS, INTERSECT). The core SQL examples use Oracle-compatible syntax. Where the sheet asks for MySQL/Oracle connectivity, a Python + MySQL example is provided. MongoDB commands are for mongosh.

Problem statement index

• 1. Banking tables, filters, aggregation, update/delete, UNION

• 2. Students and enrolments, joins, subqueries, views and indexes

• 3. Student/enrollment queries, unmatched rows, course average view

• 4. Stored procedure to categorize marks

• 5. PL/SQL function and areas table

• 6. Cursor to display employees earning above ■30,000

• 7. Library audit trigger for updates/deletes

• 8. MySQL database connectivity with CRUD/navigation

• 9. MongoDB connectivity with CRUD/navigation

• 10. Bank database queries

• 11. Bank database queries and schema changes

• 12. MongoDB employee collection

• 13. MongoDB student collection

• 14. Banking tables, set operators and loan aggregation

Problem Statement 1 — Banking tables

Create four related tables and insert 10 sample records into each. The phone column is nullable to demonstrate NULL queries.

A. Create tables

```
CREATE TABLE Customer (
  cust_id NUMBER PRIMARY KEY,
  cust_name VARCHAR2(50) NOT NULL,
  city VARCHAR2(30),
  phone VARCHAR2(15)
);

CREATE TABLE Branch (
  branch_id NUMBER PRIMARY KEY,
  branch_name VARCHAR2(40) NOT NULL,
  branch_city VARCHAR2(30)
);

CREATE TABLE Account (
  acc_no NUMBER PRIMARY KEY,
  cust_id NUMBER NOT NULL REFERENCES Customer(cust_id),
  branch_id NUMBER NOT NULL REFERENCES Branch(branch_id),
  balance NUMBER(12,2) DEFAULT 0 CHECK (balance >= 0)
);

CREATE TABLE Loan (
  loan_no NUMBER PRIMARY KEY,
  cust_id NUMBER NOT NULL REFERENCES Customer(cust_id),
  branch_id NUMBER NOT NULL REFERENCES Branch(branch_id),
  amount NUMBER(12,2) CHECK (amount >= 0)
);
```

B. Insert 10 records into each table

```
INSERT ALL
 INTO Customer VALUES (1,'Priya Shah','Pune','9876500001')
 INTO Customer VALUES (2,'Amit Patil','Mumbai','9876500002')
 INTO Customer VALUES (3,'Pooja More','Pune',NULL)
 INTO Customer VALUES (4,'Rohan Joshi','Nashik','9876500004')
 INTO Customer VALUES (5,'Neha Kulkarni','Mumbai','9876500005')
 INTO Customer VALUES (6,'Parth Desai','Pune','9876500006')
 INTO Customer VALUES (7,'Sneha Pawar','Nagpur',NULL)
 INTO Customer VALUES (8,'Om Jadhav','Mumbai','9876500008')
 INTO Customer VALUES (9,'Payal Mehta','Pune','9876500009')
 INTO Customer VALUES (10,'Kunal Patil','Kolhapur','9876500010')
SELECT 1 FROM dual;

INSERT ALL
 INTO Branch VALUES (1,'Akurdi','Pune')
 INTO Branch VALUES (2,'Chinchwad','Pune')
 INTO Branch VALUES (3,'Shivajinagar','Pune')
 INTO Branch VALUES (4,'Andheri','Mumbai')
 INTO Branch VALUES (5,'Dadar','Mumbai')
 INTO Branch VALUES (6,'Nashik Road','Nashik')
 INTO Branch VALUES (7,'Nagpur Central','Nagpur')
 INTO Branch VALUES (8,'Thane','Thane')
 INTO Branch VALUES (9,'Kolhapur','Kolhapur')
 INTO Branch VALUES (10,'Baner','Pune')
SELECT 1 FROM dual;

INSERT ALL
 INTO Account VALUES (2001,1,1,65000)
 INTO Account VALUES (2002,2,4,42000)
 INTO Account VALUES (2003,3,2,38000)
 INTO Account VALUES (2004,4,6,51000)
 INTO Account VALUES (2005,5,5,80000)
 INTO Account VALUES (2006,6,3,45000)
 INTO Account VALUES (2007,7,7,22000)
 INTO Account VALUES (2008,8,8,60000)
 INTO Account VALUES (2009,9,10,35000)
 INTO Account VALUES (2010,10,9,47000)
SELECT 1 FROM dual;

INSERT ALL
 INTO Loan VALUES (5001,1,1,150000)
 INTO Loan VALUES (5002,2,4,220000)
 INTO Loan VALUES (5003,3,2,90000)
 INTO Loan VALUES (5004,4,6,125000)
 INTO Loan VALUES (5005,5,5,300000)
```

```
 INTO Loan VALUES (5006,6,3,175000)
 INTO Loan VALUES (5007,7,7,80000)
 INTO Loan VALUES (5008,8,8,250000)
 INTO Loan VALUES (5009,9,10,110000)
 INTO Loan VALUES (5010,10,9,95000)
SELECT 1 FROM dual;

COMMIT;
```

C. Queries

```
-- 1. Customers in alphabetical order
SELECT * FROM Customer ORDER BY cust_name ASC;

-- 2. Names and cities starting with P
SELECT cust_name, city FROM Customer WHERE cust_name LIKE 'P%';

-- 3. Pune or Mumbai, with a non-NULL phone
SELECT * FROM Customer
WHERE city IN ('Pune','Mumbai') AND phone IS NOT NULL;

-- 4. Customers whose phone is NULL
SELECT * FROM Customer WHERE phone IS NULL;

-- 5. Average account balance for each branch
SELECT b.branch_name, AVG(a.balance) AS avg_balance
FROM Branch b JOIN Account a ON a.branch_id = b.branch_id
GROUP BY b.branch_name;

-- 6. Branches with average balance greater than ■40,000
SELECT b.branch_name, AVG(a.balance) AS avg_balance
FROM Branch b JOIN Account a ON a.branch_id = b.branch_id
GROUP BY b.branch_name
HAVING AVG(a.balance) > 40000;

-- 7. Show balance plus 10% interest; original balance is unchanged
SELECT acc_no, balance, balance * 1.10 AS balance_with_interest
FROM Account;

-- 8. Update a customer's phone using their name
UPDATE Customer SET phone = '9999900001'
WHERE cust_name = 'Priya Shah';

-- 9. Delete customers in Pune (dependent rows must be handled first)
DELETE FROM Loan WHERE cust_id IN (SELECT cust_id FROM Customer WHERE city='Pune');
DELETE FROM Account WHERE cust_id IN (SELECT cust_id FROM Customer WHERE city='Pune');
DELETE FROM Customer WHERE city = 'Pune';

-- 10. Customers who have an account OR a loan
SELECT cust_id FROM Account
UNION
SELECT cust_id FROM Loan;
```

For query 10, the query returns customer IDs. To display names, wrap the UNION in a subquery and join it to Customer. UNION removes duplicates.

Problem Statement 2 — Students and Enrolments

A. Create tables and insert 10 records each

```
CREATE TABLE Students (
  student_id NUMBER PRIMARY KEY,
  name VARCHAR2(50) NOT NULL,
  city VARCHAR2(30)
);

CREATE TABLE Enrolments (
  enroll_id NUMBER PRIMARY KEY,
  student_id NUMBER REFERENCES Students(student_id),
  course VARCHAR2(30) NOT NULL,
  marks NUMBER(5,2) CHECK (marks BETWEEN 0 AND 100)
);

INSERT ALL
 INTO Students VALUES (1,'Aarav','Pune')
 INTO Students VALUES (2,'Bhavna','Mumbai')
 INTO Students VALUES (3,'Chirag','Pune')
 INTO Students VALUES (4,'Diya','Nashik')
 INTO Students VALUES (5,'Eshan','Mumbai')
 INTO Students VALUES (6,'Fatima','Pune')
 INTO Students VALUES (7,'Gaurav','Nagpur')
 INTO Students VALUES (8,'Isha','Pune')
 INTO Students VALUES (9,'Kabir','Thane')
 INTO Students VALUES (10,'Meera','Mumbai')
SELECT 1 FROM dual;

INSERT ALL
 INTO Enrolments VALUES (101,1,'DBMS',88)
 INTO Enrolments VALUES (102,2,'Python',76)
 INTO Enrolments VALUES (103,3,'DBMS',95)
 INTO Enrolments VALUES (104,4,'Java',68)
 INTO Enrolments VALUES (105,5,'Python',84)
 INTO Enrolments VALUES (106,6,'DBMS',91)
 INTO Enrolments VALUES (107,7,'Java',72)
 INTO Enrolments VALUES (108,8,'Web Development',79)
 INTO Enrolments VALUES (109,9,'DBMS',62)
 INTO Enrolments VALUES (110,3,'Python',89)
SELECT 1 FROM dual;
COMMIT;
```

B. Queries, view and indexes

```
-- 1. Student names, course names and marks
SELECT s.name, e.course, e.marks
FROM Students s JOIN Enrolments e ON e.student_id = s.student_id;

-- 2. Include students not enrolled in a course
SELECT s.name, e.course, e.marks
FROM Students s LEFT JOIN Enrolments e ON e.student_id = s.student_id;

-- 3. All enrollments with corresponding student names
SELECT e.*, s.name
FROM Enrolments e LEFT JOIN Students s ON s.student_id = e.student_id;

-- 4. All possible combinations of students and distinct courses (Cartesian product)
SELECT s.name, c.course
FROM Students s CROSS JOIN (SELECT DISTINCT course FROM Enrolments) c;

-- 5. Students whose enrollment marks exceed the average of all enrollments
SELECT DISTINCT s.student_id, s.name
FROM Students s JOIN Enrolments e ON e.student_id=s.student_id
WHERE e.marks > (SELECT AVG(marks) FROM Enrolments);

-- 6. Students who obtained marks greater than 80
SELECT DISTINCT s.student_id, s.name, e.marks
FROM Students s JOIN Enrolments e ON e.student_id=s.student_id
WHERE e.marks > 80;

-- 7. Students enrolled in DBMS
SELECT DISTINCT s.student_id, s.name
FROM Students s JOIN Enrolments e ON e.student_id=s.student_id
WHERE e.course='DBMS';

-- 8. Student ID(s) with the highest marks
SELECT student_id FROM Enrolments
WHERE marks = (SELECT MAX(marks) FROM Enrolments);
```

```
-- 9. Create and use view; update marks through the view
CREATE OR REPLACE VIEW student_course_view AS
SELECT s.student_id, s.name, e.enroll_id, e.course, e.marks
FROM Students s JOIN Enrolments e ON e.student_id=s.student_id;

SELECT * FROM student_course_view;
UPDATE student_course_view SET marks=90 WHERE enroll_id=101;

-- 10. Index on name and unique index on enroll_id
CREATE INDEX idx_students_name ON Students(name);
CREATE UNIQUE INDEX uq_enrolments_enroll_id ON Enrolments(enroll_id);
```

The view includes primary keys so Oracle can identify the underlying row for the update. A join view may be updatable only under Oracle's rules; if an update is rejected, update Enrolments directly using enroll_id.

Problem Statement 3 — Student and course queries

Use the Students and Enrolments tables created in Problem Statement 2. The wording in the sheet says “four tables” but lists only these two tables, so this solution follows the listed schema.

```
-- 1. Student names and courses
SELECT s.name, e.course
FROM Students s JOIN Enrolments e ON s.student_id=e.student_id;

-- 2. All students, including those not enrolled
SELECT s.name, e.course, e.marks
FROM Students s LEFT JOIN Enrolments e ON s.student_id=e.student_id;

-- 3. Students enrolled in DBMS and their marks
SELECT s.name, e.marks
FROM Students s JOIN Enrolments e ON s.student_id=e.student_id
WHERE e.course='DBMS';

-- 4. Enrollments without a matching student (normally none due to FK)
SELECT e.*
FROM Enrolments e LEFT JOIN Students s ON s.student_id=e.student_id
WHERE s.student_id IS NULL;

-- 5. Students with marks below the average of all enrollments
SELECT DISTINCT s.student_id, s.name, e.marks
FROM Students s JOIN Enrolments e ON s.student_id=e.student_id
WHERE e.marks < (SELECT AVG(marks) FROM Enrolments);

-- 6. Students with the highest marks
SELECT s.student_id, s.name, e.marks
FROM Students s JOIN Enrolments e ON s.student_id=e.student_id
WHERE e.marks=(SELECT MAX(marks) FROM Enrolments);

-- 7. Students who belong to Pune
SELECT * FROM Students WHERE city='Pune';

-- 8. Students enrolled in Python
SELECT DISTINCT s.student_id, s.name
FROM Students s JOIN Enrolments e ON s.student_id=e.student_id
WHERE e.course='Python';

-- 9. Course average view
CREATE OR REPLACE VIEW course_average_view AS
SELECT course, AVG(marks) AS average_marks
FROM Enrolments GROUP BY course;
SELECT * FROM course_average_view;

-- 10. Index on city and unique index on student_id
CREATE INDEX idx_students_city ON Students(city);
CREATE UNIQUE INDEX uq_students_student_id ON Students(student_id);
```

Query 4 is a diagnostic query: with an enabled foreign key, orphan enrolments cannot normally be inserted. A unique index on student_id is redundant because it is already the primary key, but it satisfies the requested exercise if the PK constraint is removed or the index name is changed appropriately.

Problem Statement 4 — Stored procedure proc_Grade

The sheet specifies: 990–1500 = Distinction; 900–989 = First Class; 825–899 = Higher Second Class. It does not define categories below 825 or above 1500, so the procedure labels them “Other”.

```
CREATE TABLE Stud_Marks (
  rollno NUMBER PRIMARY KEY,
  name VARCHAR2(50),
  total_marks NUMBER(5)
);

CREATE TABLE Result (
  Roll NUMBER PRIMARY KEY,
  Name VARCHAR2(50),
  Class VARCHAR2(30)
);

INSERT ALL
 INTO Stud_Marks VALUES (1,'Aarav',1200)
 INTO Stud_Marks VALUES (2,'Bhavna',995)
 INTO Stud_Marks VALUES (3,'Chirag',970)
 INTO Stud_Marks VALUES (4,'Diya',910)
 INTO Stud_Marks VALUES (5,'Eshan',850)
 INTO Stud_Marks VALUES (6,'Fatima',830)
 INTO Stud_Marks VALUES (7,'Gaurav',800)
 INTO Stud_Marks VALUES (8,'Isha',1000)
 INTO Stud_Marks VALUES (9,'Kabir',890)
 INTO Stud_Marks VALUES (10,'Meera',700)
SELECT 1 FROM dual;
COMMIT;

CREATE OR REPLACE PROCEDURE proc_Grade IS
  v_class VARCHAR2(30);
BEGIN
  DELETE FROM Result;

  FOR r IN (SELECT rollno, name, total_marks FROM Stud_Marks) LOOP
    IF r.total_marks BETWEEN 990 AND 1500 THEN
      v_class := 'Distinction';
    ELSIF r.total_marks BETWEEN 900 AND 989 THEN
      v_class := 'First Class';
    ELSIF r.total_marks BETWEEN 825 AND 899 THEN
      v_class := 'Higher Second Class';
    ELSE
      v_class := 'Other';
    END IF;

    INSERT INTO Result(Roll, Name, Class)
    VALUES (r.rollno, r.name, v_class);
  END LOOP;

  COMMIT;
END;
/

BEGIN
  proc_Grade;
END;
/

SELECT * FROM Result ORDER BY Roll;
```

If the practical expects only the procedure and not sample records, execute the CREATE PROCEDURE block after creating and populating both tables.

Problem Statement 5 — Function to calculate circle area

```
CREATE TABLE areas (
  radius NUMBER(5,2),
  area NUMBER(12,4)
);

CREATE OR REPLACE FUNCTION circle_area(p_radius NUMBER)
RETURN NUMBER IS
BEGIN
  RETURN 3.141592653589793 * p_radius * p_radius;
END;
/

BEGIN
  DELETE FROM areas;
  FOR r IN 5..9 LOOP
    INSERT INTO areas(radius, area)
    VALUES (r, circle_area(r));
  END LOOP;
  COMMIT;
END;
/

SELECT * FROM areas ORDER BY radius;
```

Formula: Area = π × radius × radius. The function returns the calculated area; the PL/SQL loop stores values for radii 5, 6, 7, 8 and 9.

Problem Statement 6 — Cursor for employees earning above ■30,000

```
CREATE TABLE Employee (
  Emp_ID NUMBER PRIMARY KEY,
  Emp_Name VARCHAR2(50) NOT NULL,
  Department VARCHAR2(40),
  Salary NUMBER(10,2)
);

INSERT ALL
 INTO Employee VALUES (1,'Aarav','Computer',45000)
 INTO Employee VALUES (2,'Bhavna','IT',28000)
 INTO Employee VALUES (3,'Chirag','HR',32000)
 INTO Employee VALUES (4,'Diya','Computer',52000)
 INTO Employee VALUES (5,'Eshan','Finance',30000)
 INTO Employee VALUES (6,'Fatima','IT',39000)
 INTO Employee VALUES (7,'Gaurav','HR',27000)
 INTO Employee VALUES (8,'Isha','Computer',61000)
 INTO Employee VALUES (9,'Kabir','Finance',35000)
 INTO Employee VALUES (10,'Meera','IT',25000)
SELECT 1 FROM dual;
COMMIT;

SET SERVEROUTPUT ON;

DECLARE
  CURSOR c_emp IS
    SELECT Emp_ID, Emp_Name, Department, Salary
    FROM Employee
    WHERE Salary > 30000
    ORDER BY Emp_ID;
BEGIN
  FOR r IN c_emp LOOP
    DBMS_OUTPUT.PUT_LINE(
      'ID: ' || r.Emp_ID ||
      ', Name: ' || r.Emp_Name ||
      ', Department: ' || r.Department ||
      ', Salary: ' || r.Salary
    );
  END LOOP;
END;
/
```

A cursor processes query rows one at a time. The cursor FOR loop automatically opens the cursor, fetches each row, and closes it.

Problem Statement 7 — Library audit trigger

```
CREATE TABLE Library (
  book_id NUMBER PRIMARY KEY,
  book_name VARCHAR2(100),
  author VARCHAR2(80),
  category VARCHAR2(40),
  date_added DATE,
  operation VARCHAR2(10)
);

CREATE TABLE Library_Audit (
  book_id NUMBER,
  book_name VARCHAR2(100),
  author VARCHAR2(80),
  category VARCHAR2(40),
  date_added DATE,
  operation VARCHAR2(10),
  audit_time TIMESTAMP DEFAULT SYSTIMESTAMP
);

INSERT ALL
 INTO Library VALUES (1,'DBMS Basics','K. Rao','Education',DATE '2024-01-10','INSERT')
 INTO Library VALUES (2,'SQL Made Easy','P. Shah','Education',DATE '2024-02-11','INSERT')
 INTO Library VALUES (3,'Oracle Guide','R. Mehta','Technology',DATE '2024-03-12','INSERT')
 INTO Library VALUES (4,'Python Intro','S. Patil','Technology',DATE '2024-04-13','INSERT')
 INTO Library VALUES (5,'Networks','A. Joshi','Technology',DATE '2024-05-14','INSERT')
 INTO Library VALUES (6,'Clean Code','R. Martin','Technology',DATE '2024-06-15','INSERT')
 INTO Library VALUES (7,'Algorithms','T. Cormen','Education',DATE '2024-07-16','INSERT')
 INTO Library VALUES (8,'Operating Systems','A. Silberschatz','Education',DATE '2024-08-17','INSERT')
 INTO Library VALUES (9,'Web Development','M. More','Technology',DATE '2024-09-18','INSERT')
 INTO Library VALUES (10,'Data Science','N. Kulkarni','Technology',DATE '2024-10-19','INSERT')
SELECT 1 FROM dual;
COMMIT;

CREATE OR REPLACE TRIGGER trg_library_audit
BEFORE UPDATE OR DELETE ON Library
FOR EACH ROW
BEGIN
  IF UPDATING THEN
    INSERT INTO Library_Audit
      (book_id, book_name, author, category, date_added, operation)
    VALUES
      (:OLD.book_id, :OLD.book_name, :OLD.author, :OLD.category,
       :OLD.date_added, 'UPDATE');
  ELSIF DELETING THEN
    INSERT INTO Library_Audit
      (book_id, book_name, author, category, date_added, operation)
    VALUES
      (:OLD.book_id, :OLD.book_name, :OLD.author, :OLD.category,
       :OLD.date_added, 'DELETE');
  END IF;
END;
/

-- Test the trigger
UPDATE Library SET category='Reference' WHERE book_id=1;
DELETE FROM Library WHERE book_id=2;
SELECT * FROM Library_Audit ORDER BY audit_time;
```

A row-level BEFORE trigger copies the old values (:OLD) before the original row is updated or deleted. The audit table records which operation occurred.

Problem Statement 8 — MySQL connectivity with CRUD

Example implementation: Python + MySQL Connector. Install with **pip install mysql-connector-python**, create the table in MySQL, then run the Python program. Replace credentials with your own local database credentials.

A. MySQL table

```
CREATE DATABASE IF NOT EXISTS college_db;
USE college_db;

CREATE TABLE IF NOT EXISTS employee (
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50) NOT NULL,
  department VARCHAR(40),
  salary DECIMAL(10,2)
);
```

B. Python program (add, view, edit, delete)

```
import mysql.connector

con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="YOUR_PASSWORD",
    database="college_db"
)
cur = con.cursor()

def add_employee():
    emp_id = int(input("Employee ID: "))
    name = input("Name: ")
    dept = input("Department: ")
    salary = float(input("Salary: "))
    cur.execute(
        "INSERT INTO employee VALUES (%s, %s, %s, %s)",
        (emp_id, name, dept, salary)
    )
    con.commit()
    print("Employee added.")

def view_employees():
    cur.execute("SELECT * FROM employee ORDER BY emp_id")
    for row in cur.fetchall():
        print(row)

def edit_employee():
    emp_id = int(input("ID to edit: "))
    name = input("New name: ")
    dept = input("New department: ")
    salary = float(input("New salary: "))
    cur.execute(
        "UPDATE employee SET emp_name=%s, department=%s, salary=%s "
        "WHERE emp_id=%s",
        (name, dept, salary, emp_id)
    )
    con.commit()
    print("Rows updated:", cur.rowcount)

def delete_employee():
    emp_id = int(input("ID to delete: "))
    cur.execute("DELETE FROM employee WHERE emp_id=%s", (emp_id,))
    con.commit()
    print("Rows deleted:", cur.rowcount)

while True:
    print("\n1.Add 2.View 3.Edit 4.Delete 5.Exit")
    choice = input("Choose: ")
    if choice == "1":
        add_employee()
    elif choice == "2":
        view_employees()
    elif choice == "3":
        edit_employee()
    elif choice == "4":
        delete_employee()
    elif choice == "5":
        break
    else:
```

```
        print("Invalid choice")

cur.close()
con.close()
```

Use parameterized queries (%s placeholders) rather than building SQL by string concatenation. This helps prevent SQL injection.

Problem Statement 9 — MongoDB connectivity with CRUD

Example implementation: Python + PyMongo. Install with **pip install pymongo** and ensure MongoDB is running locally.

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["ICEM"]
employees = db["employee"]

def add_employee():
    doc = {
        "empid": int(input("Employee ID: ")),
        "ename": input("Name: "),
        "dept": input("Department: "),
        "salary": float(input("Salary: "))
    }
    employees.insert_one(doc)
    print("Employee added.")

def view_employees():
    for doc in employees.find({}, {"_id": 0}):
        print(doc)

def edit_employee():
    empid = int(input("Employee ID to edit: "))
    result = employees.update_one(
        {"empid": empid},
        {"$set": {
            "ename": input("New name: "),
            "dept": input("New department: "),
            "salary": float(input("New salary: "))
        }}
    )
    print("Matched:", result.matched_count, "Modified:", result.modified_count)

def delete_employee():
    empid = int(input("Employee ID to delete: "))
    result = employees.delete_one({"empid": empid})
    print("Deleted:", result.deleted_count)

while True:
    print("\n1.Add 2.View 3.Edit 4.Delete 5.Exit")
    choice = input("Choose: ")
    if choice == "1":
        add_employee()
    elif choice == "2":
        view_employees()
    elif choice == "3":
        edit_employee()
    elif choice == "4":
        delete_employee()
    elif choice == "5":
        break
    else:
        print("Invalid choice")

client.close()
```

Problem Statement 10 — Bank database

A. Create tables with constraints

```
CREATE TABLE Branch (
  branch_name VARCHAR2(40) PRIMARY KEY,
  branch_city VARCHAR2(40) NOT NULL,
  assets NUMBER(14,2) NOT NULL
);

CREATE TABLE Customer (
  cust_name VARCHAR2(50) PRIMARY KEY,
  cust_street VARCHAR2(80),
  cust_city VARCHAR2(40)
);

CREATE TABLE Account (
  acc_no NUMBER PRIMARY KEY,
  branch_name VARCHAR2(40) REFERENCES Branch(branch_name),
  balance NUMBER(12,2) CHECK (balance >= 0)
);

CREATE TABLE Depositor (
  cust_name VARCHAR2(50) REFERENCES Customer(cust_name),
  acc_no NUMBER REFERENCES Account(acc_no),
  CONSTRAINT pk_depositor PRIMARY KEY (cust_name, acc_no)
);

CREATE TABLE Loan (
  loan_no NUMBER PRIMARY KEY,
  branch_name VARCHAR2(40) REFERENCES Branch(branch_name),
  amount NUMBER(12,2) CHECK (amount >= 0)
);

CREATE TABLE Borrower (
  cust_name VARCHAR2(50) REFERENCES Customer(cust_name),
  loan_no NUMBER REFERENCES Loan(loan_no),
  CONSTRAINT pk_borrower PRIMARY KEY (cust_name, loan_no)
);
```

B. Insert suitable data

```
INSERT ALL
 INTO Branch VALUES ('Akurdi','Pune',9000000)
 INTO Branch VALUES ('Chinchwad','Pune',8000000)
 INTO Branch VALUES ('Shivajinagar','Pune',12000000)
 INTO Branch VALUES ('Andheri','Mumbai',15000000)
 INTO Branch VALUES ('Dadar','Mumbai',11000000)
SELECT 1 FROM dual;

INSERT ALL
 INTO Customer VALUES ('Amit Patil','MG Road','Pune')
 INTO Customer VALUES ('Bhavna Shah','FC Road','Pune')
 INTO Customer VALUES ('Chirag Mehta','Link Road','Mumbai')
 INTO Customer VALUES ('Diya More','Station Road','Nashik')
 INTO Customer VALUES ('Eshan Joshi','Park Street','Mumbai')
 INTO Customer VALUES ('Fatima Khan',NULL,'Pune')
 INTO Customer VALUES ('Gaurav Pawar','Main Road','Nagpur')
 INTO Customer VALUES ('Isha Desai','Lake Road','Mumbai')
 INTO Customer VALUES ('Kabir Rao','Market Road','Pune')
 INTO Customer VALUES ('Meera Kulkarni','College Road','Thane')
SELECT 1 FROM dual;

INSERT ALL
 INTO Account VALUES (3001,'Akurdi',25000)
 INTO Account VALUES (3002,'Akurdi',42000)
 INTO Account VALUES (3003,'Chinchwad',18000)
 INTO Account VALUES (3004,'Chinchwad',36000)
 INTO Account VALUES (3005,'Shivajinagar',50000)
 INTO Account VALUES (3006,'Andheri',65000)
 INTO Account VALUES (3007,'Andheri',30000)
 INTO Account VALUES (3008,'Dadar',48000)
 INTO Account VALUES (3009,'Dadar',52000)
 INTO Account VALUES (3010,'Akurdi',15000)
SELECT 1 FROM dual;

INSERT ALL
 INTO Depositor VALUES ('Amit Patil',3001)
 INTO Depositor VALUES ('Bhavna Shah',3002)
 INTO Depositor VALUES ('Chirag Mehta',3003)
```

```
 INTO Depositor VALUES ('Diya More',3004)
 INTO Depositor VALUES ('Eshan Joshi',3005)
 INTO Depositor VALUES ('Fatima Khan',3006)
 INTO Depositor VALUES ('Gaurav Pawar',3007)
 INTO Depositor VALUES ('Isha Desai',3008)
 INTO Depositor VALUES ('Kabir Rao',3009)
 INTO Depositor VALUES ('Meera Kulkarni',3010)
SELECT 1 FROM dual;

INSERT ALL
 INTO Loan VALUES (6001,'Akurdi',15000)
 INTO Loan VALUES (6002,'Akurdi',45000)
 INTO Loan VALUES (6003,'Chinchwad',125000)
 INTO Loan VALUES (6004,'Chinchwad',350000)
 INTO Loan VALUES (6005,'Shivajinagar',600000)
 INTO Loan VALUES (6006,'Andheri',750000)
 INTO Loan VALUES (6007,'Andheri',550000)
 INTO Loan VALUES (6008,'Dadar',250000)
 INTO Loan VALUES (6009,'Dadar',500000)
 INTO Loan VALUES (6010,'Akurdi',9000)
SELECT 1 FROM dual;

INSERT ALL
 INTO Borrower VALUES ('Amit Patil',6001)
 INTO Borrower VALUES ('Bhavna Shah',6002)
 INTO Borrower VALUES ('Chirag Mehta',6003)
 INTO Borrower VALUES ('Diya More',6004)
 INTO Borrower VALUES ('Eshan Joshi',6005)
 INTO Borrower VALUES ('Fatima Khan',6006)
 INTO Borrower VALUES ('Gaurav Pawar',6007)
 INTO Borrower VALUES ('Isha Desai',6008)
 INTO Borrower VALUES ('Kabir Rao',6009)
 INTO Borrower VALUES ('Meera Kulkarni',6010)
SELECT 1 FROM dual;
COMMIT;
```

C. Queries

```
-- 2. Add phone number column
ALTER TABLE Customer ADD (phone VARCHAR2(15));

-- 3. Delete phone number attribute
ALTER TABLE Customer DROP COLUMN phone;

-- 4. Branch names in Loan relation
SELECT DISTINCT branch_name FROM Loan;

-- 5. Customers who have a loan: name, loan number and amount
SELECT c.cust_name, l.loan_no, l.amount
FROM Customer c
JOIN Borrower b ON b.cust_name=c.cust_name
JOIN Loan l ON l.loan_no=b.loan_no;

-- 6. Alphabetical list of customers with a loan from Akurdi
SELECT DISTINCT c.cust_name
FROM Customer c
JOIN Borrower b ON b.cust_name=c.cust_name
JOIN Loan l ON l.loan_no=b.loan_no
WHERE l.branch_name='Akurdi'
ORDER BY c.cust_name;

-- 7. Average account balance at Chinchwad
SELECT AVG(balance) AS avg_balance
FROM Account WHERE branch_name='Chinchwad';

-- 8. Number of depositors at each branch
SELECT a.branch_name, COUNT(DISTINCT d.cust_name) AS depositor_count
FROM Account a JOIN Depositor d ON d.acc_no=a.acc_no
GROUP BY a.branch_name;

-- 9. Total loan amount for each branch
SELECT branch_name, SUM(amount) AS total_loan
FROM Loan GROUP BY branch_name;

-- 10. Branches whose average loan amount > ■500,000
SELECT branch_name, AVG(amount) AS avg_loan
FROM Loan GROUP BY branch_name
HAVING AVG(amount) > 500000;
```

The sheet's question 3 removes the phone column immediately after adding it; run those ALTER TABLE statements in order if demonstrating both operations. If your Oracle version does not permit dropping a column due to dependencies, check for

constraints or indexes first.

Problem Statement 11 — Bank queries and alterations

Use the bank tables and sample data from Problem Statement 10. Run each statement in order; this statement asks for a schema change and data queries.

```
-- 1. Rename assets to Property
ALTER TABLE Branch RENAME COLUMN assets TO Property;

-- 2. Loan numbers from Akurdi with amount > 12,000
SELECT loan_no
FROM Loan
WHERE branch_name='Akurdi' AND amount > 12000;

-- 3. Average account balance at each branch
SELECT branch_name, AVG(balance) AS avg_balance
FROM Account GROUP BY branch_name;

-- 4. Branches with average balance > 12,000
SELECT branch_name, AVG(balance) AS avg_balance
FROM Account
GROUP BY branch_name
HAVING AVG(balance) > 12000;

-- 5. Number of tuples (rows) in Customer
SELECT COUNT(*) AS customer_count FROM Customer;

-- 6. Total loan amount given by bank
SELECT SUM(amount) AS total_loan_amount FROM Loan;

-- 7. Delete loans with amount between 1,300 and 1,500
DELETE FROM Borrower
WHERE loan_no IN (
  SELECT loan_no FROM Loan WHERE amount BETWEEN 1300 AND 1500
);
DELETE FROM Loan WHERE amount BETWEEN 1300 AND 1500;

-- 8. Customers whose street address is not available
SELECT * FROM Customer WHERE cust_street IS NULL;

-- 9. Branches where branch city is available
SELECT * FROM Branch WHERE branch_city IS NOT NULL;

-- 10. Customers whose names start with A
SELECT * FROM Customer WHERE cust_name LIKE 'A%';
```

Query 7 deletes Borrower rows first to preserve the foreign-key constraint. BETWEEN includes both endpoints.

Problem Statement 12 — MongoDB employee collection

```
use ICEM

db.employee.insertMany([
  {empid:1, ename:"Aarav", dept:"Computer", salary:45000},
  {empid:2, ename:"Bhavna", dept:"IT", salary:28000},
  {empid:3, ename:"Chirag", dept:"HR", salary:32000},
  {empid:4, ename:"Diya", dept:"Computer", salary:52000},
  {empid:5, ename:"Eshan", dept:"Finance", salary:30000},
  {empid:6, ename:"Fatima", dept:"IT", salary:39000},
  {empid:7, ename:"Gaurav", dept:"HR", salary:27000},
  {empid:8, ename:"Isha", dept:"Computer", salary:61000},
  {empid:9, ename:"Kabir", dept:"Finance", salary:35000},
  {empid:10, ename:"Meera", dept:"IT", salary:25000}
])
```

Queries

```
-- 1. All employees
db.employee.find()

-- 2. Computer department
db.employee.find({dept:"Computer"})

-- 3. Computer OR IT
db.employee.find({$or:[{dept:"Computer"},{dept:"IT"}]})

-- 4. Salary greater than 30,000
db.employee.find({salary:{$gt:30000}})

-- 5. Salary >= 40,000 AND Computer department
db.employee.find({salary:{$gte:40000}, dept:"Computer"})

-- 6. Create and drop index on ename
db.employee.createIndex({ename:1})
db.employee.dropIndex("ename_1")

-- 7. Salary between 20,000 and 40,000 (inclusive)
db.employee.find({salary:{$gte:20000,$lte:40000}})

-- 8. Department is not IT
db.employee.find({dept:{$ne:"IT"}})

-- 9. Total salary in each department
db.employee.aggregate([
  {$group:{_id:"$dept", totalSalary:{$sum:"$salary"}}}
])

-- 10. Average salary in each department
db.employee.aggregate([
  {$group:{_id:"$dept", averageSalary:{$avg:"$salary"}}}
])
```

MongoDB uses $gt (greater than), $gte (greater than or equal), $ne (not equal), $or (logical OR), and aggregation stages such as $group.

Problem Statement 13 — MongoDB student collection

```
use Student

db.student.insertMany([
  {roll_no:1, sname:"Aarav", dept:"Computer", marks:88},
  {roll_no:2, sname:"Bhavna", dept:"IT", marks:76},
  {roll_no:3, sname:"Chirag", dept:"Computer", marks:95},
  {roll_no:4, sname:"Diya", dept:"ENTC", marks:68},
  {roll_no:5, sname:"Eshan", dept:"IT", marks:84},
  {roll_no:6, sname:"Fatima", dept:"Computer", marks:91},
  {roll_no:7, sname:"Gaurav", dept:"ENTC", marks:32},
  {roll_no:8, sname:"Isha", dept:"Computer", marks:79},
  {roll_no:9, sname:"Kabir", dept:"IT", marks:62},
  {roll_no:10, sname:"Meera", dept:"ENTC", marks:40}
])
```

Queries

```
-- 1. All students
db.student.find()

-- 2. Computer department
db.student.find({dept:"Computer"})

-- 3. Computer OR IT
db.student.find({$or:[{dept:"Computer"},{dept:"IT"}]})

-- 4. Marks greater than 70
db.student.find({marks:{$gt:70}})

-- 5. Marks >= 80 AND Computer department
db.student.find({marks:{$gte:80}, dept:"Computer"})

-- 6. Marks < 40 OR department is IT
db.student.find({$or:[{marks:{$lt:40}},{dept:"IT"}]})

-- 7. Marks between 40 and 80 inclusive
db.student.find({marks:{$gte:40,$lte:80}})

-- 8. Department is not IT
db.student.find({dept:{$ne:"IT"}})

-- 9. Total marks in each department
db.student.aggregate([
  {$group:{_id:"$dept", totalMarks:{$sum:"$marks"}}}
])

-- 10. Create and drop index on sname
db.student.createIndex({sname:1})
db.student.dropIndex("sname_1")
```

Problem Statement 14 — Banking queries and set operators

Use the four-table banking schema and data from Problem Statement 1. The set-operator queries use Oracle's INTERSECT and MINUS, and compare customer IDs to avoid duplicate names.

```
-- 1. Customers in Pune or Mumbai, alphabetically
SELECT * FROM Customer
WHERE city IN ('Pune','Mumbai')
ORDER BY cust_name;

-- 2. Names containing the letter 'a' (case-sensitive behavior depends on collation)
SELECT * FROM Customer
WHERE LOWER(cust_name) LIKE '%a%';

-- 3. Customers whose phone is not NULL
SELECT * FROM Customer WHERE phone IS NOT NULL;

-- 4. Pune customers whose names start with P, OR all Mumbai customers
SELECT * FROM Customer
WHERE (city='Pune' AND cust_name LIKE 'P%')
   OR city='Mumbai';

-- 5. Maximum and average loan amount for each branch
SELECT b.branch_name, MAX(l.amount) AS max_loan,
       AVG(l.amount) AS avg_loan
FROM Branch b JOIN Loan l ON l.branch_id=b.branch_id
GROUP BY b.branch_name;

-- 6. Branches with total loan amount > ■1,00,000
SELECT b.branch_name, SUM(l.amount) AS total_loan
FROM Branch b JOIN Loan l ON l.branch_id=b.branch_id
GROUP BY b.branch_name
HAVING SUM(l.amount) > 100000;

-- 7. Show balance plus ■500 without modifying stored balance
SELECT acc_no, balance, balance + 500 AS balance_plus_500
FROM Account;

-- 8. Change branch for account 2005 (example: branch_id 5 to branch_id 4)
UPDATE Account SET branch_id=4 WHERE acc_no=2005;

-- 9. Delete customers whose phone is NULL (delete dependent rows first)
DELETE FROM Loan
WHERE cust_id IN (SELECT cust_id FROM Customer WHERE phone IS NULL);
DELETE FROM Account
WHERE cust_id IN (SELECT cust_id FROM Customer WHERE phone IS NULL);
DELETE FROM Customer WHERE phone IS NULL;

-- 10(a). Customers who have both an account and a loan
SELECT cust_id FROM Account
INTERSECT
SELECT cust_id FROM Loan;

-- 10(b). Customers who have an account but no loan
SELECT cust_id FROM Account
MINUS
SELECT cust_id FROM Loan;
```

If you need customer names for 10(a) or 10(b), place each set operation inside a subquery and join the resulting cust_id values to Customer. Query 8 assumes branch_id 4 is the desired different branch; choose another valid branch_id if your faculty specifies one.

Quick revision — key SQL and PL/SQL concepts

Concept	Meaning / example
DDL	Defines structure: CREATE, ALTER, DROP, TRUNCATE.
DML	Changes data: INSERT, UPDATE, DELETE. SELECT is often categorized as DQL.
Primary key	Uniquely identifies each row; cannot be NULL.
Foreign key	Maintains a valid relationship to a key in another table.
WHERE	Filters individual rows before grouping.
GROUP BY	Groups rows for aggregate functions such as AVG, SUM, COUNT.
HAVING	Filters groups after GROUP BY, e.g. HAVING AVG(balance) > 40000.
JOIN	Combines related rows from tables.
LEFT JOIN	Keeps every row from the left table even without a match.
UNION	Combines result sets and removes duplicates.
INTERSECT	Returns rows common to both result sets.
MINUS	Oracle set operator returning rows in first result but not second.
View	A named query presented like a virtual table.
Index	Can speed up lookups; uses extra storage and may slow writes.
PL/SQL	Oracle's procedural extension to SQL; supports variables, loops, conditions, procedures, functions and exceptions.
Cursor	Allows a query's result rows to be processed one at a time.
Trigger	Automatically runs when a specified database event occurs.
Stored procedure	A named stored PL/SQL program that performs an operation.
Function	A stored program that returns a value.
MongoDB find	Reads matching documents, e.g. db.employee.find({dept:'IT'}).
MongoDB aggregation	Processes documents in stages; $group can calculate $sum and $avg.

Practical exam tip: explain the purpose of each clause briefly. For example, GROUP BY forms one group per branch, AVG(balance) calculates each group's average, and HAVING filters those groups based on the aggregate result.